

hw__4

January 5, 2026

```
[1]: from ProblemiInversi import operators, solvers, utilities
import numpy as np
from numpy.linalg import svd
import matplotlib.pyplot as plt
from skimage import data
```

0.1 Compressione file SVD

```
[2]: X=plt.imread("lena.bmp")

nx, ny = X.shape
rank=min(nx,ny)

U, s, Vt = svd(X, full_matrices=False)

p_arr = np.arange(1,150)

norm_A = np.linalg.norm(X)

for i in p_arr:

    S_p = np.diag(s[:i])
    A_p = U[:, :i] @ S_p @ Vt[:i, :]

    diff_norm = np.linalg.norm(X - A_p)
    rel_error = diff_norm / norm_A

    cp = (1/i) * rank - 1
```

0.2 Sfocamento dell'Immagine

```
[6]: k=11
kernel = utilities.gaussian2d_kernel(k=k, sigma=3.5)
A = operators.ConvolutionOperator(kernel)

y = A(X)
```

```

noise_level = 0.05

y_delta = y + utilities.gaussian_noise(y, noise_level=noise_level)

ER_sfocatura = utilities.rel_err(y_delta, X)
ER_sfocatura = np.round(ER_sfocatura, 4)

```

0.3 Soluzione Naive

```

[4]: cgl_slover = solvers.CGLS(A)

x0 = np.zeros_like(X)
kmax = 30
tolf = 1e-8
tolx = 1e-8

x_cgls = cgl_slover.solve(y_delta, x0, kmax, tolf, tolx)

ER_naive=np.round(utilities.rel_err(x_cgls, X), 5)

```

0.4 Ricostruzione Tikhonov

```

[7]: def Tikhonov(A, y, lamb, y_delta, X):
    L = operators.Identity()
    M = operators.TikhonovOperator(A, L, lamb)

    ybar_delta = np.pad(y_delta, ((0, nx), (0, 0)))

    cgl_slover = solvers.CGLS(M)
    x_tik = cgl_slover.solve(ybar_delta, x0, kmax, tolf, tolx)

    abs_err = np.linalg.norm(x_tik - X)

    residual_vector = A(x_tik) - y_delta
    res_val = np.linalg.norm(residual_vector)

    return x_tik, abs_err, res_val

lambdas = np.logspace(-5, 1, 20)

results = []

y = A(X)

```

```

for i in lambdas:
    results.append(Tikhonov(A, y, i, y_delta, X))

solutions, abs_error, residuals = zip(*results)

noise_norm = np.linalg.norm(y_delta - y)

idx_min_err = np.argmin(abs_error)
best_lambda_err = lambdas[idx_min_err]
min_error_val = abs_error[idx_min_err]

print(f"\nLambda (Minimo Errore): {best_lambda_err:.2e}")
print(f"Errore minimo: {min_error_val:.4f}")

idx_discrepancy = np.argmin(np.abs(np.array(residuals) - noise_norm))
best_lambda_discrepancy = lambdas[idx_discrepancy]

print(f"Lambda (Discrepanza): {best_lambda_discrepancy:.2e}")
print(f"Residuo al lambda discrepanza: {residuals[idx_discrepancy]:.4f}")

```

```

Lambda (Minimo Errore): 1.27e-01
Errore minimo: 6094.2222
Lambda (Discrepanza): 1.27e-01
Residuo al lambda discrepanza: 3453.6538

```

0.5 Ricostruzione Con Total Variation

```

[13]: lambdas_tv = np.logspace(-4, 2, 20)

errors_tv = []
solutions_tv = []

beta = 1e-3
gd_tv_solver = solvers.GDTotalVariation(A, beta=beta)

x0 = np.zeros_like(X)
kmax = 50
tolf = 1e-8
tolx = 1e-8

for lmbda in lambdas_tv:
    val, _, _ = gd_tv_solver.solve(y_delta, lmbda, x0, kmax, tolf, tolx)
    err = np.linalg.norm(val - X)
    errors_tv.append(err)

```

```
solutions_tv.append(val)
```

```
[12]: idx_min_tv = np.argmin(errors_tv)

best_lambda_tv = lambdas_tv[idx_min_tv]
x_best_tv = solutions_tv[idx_min_tv]
min_err_tv = errors_tv[idx_min_tv]

print(f"\nMiglior Lambda TV: {best_lambda_tv:.2e}")
print(f"Errore Minimo TV: {min_err_tv:.4f}")

ssim_x = utilities.ssim(x_best_tv, X)
psnr_x = utilities.psnr(x_best_tv, X)
```

```
-----
NameError                                Traceback (most recent call last)
Cell In[12], line 1
----> 1 idx_min_tv = np.argmin(errors_tv)
      3 best_lambda_tv = lambdas_tv[idx_min_tv]
      4 x_best_tv = solutions_tv[idx_min_tv]

NameError: name 'errors_tv' is not defined
```

Ecco la ricostruzione con Total Variation avendo λ con l'errore minore: